Riepilogo — L07: strutture dati complesse e iteratori

Tutto quello che si fa con stringhe, tuple, liste e dizionari, in un posto solo: gli operatori, i metodi, le funzioni predefinite. Per ogni operazione c'è un esempio e il suo risultato, eseguiti con Python 3.14.

Comprende anche alcuni metodi che i lucidi non mostrano, come `replace`, `lower`, `insert` e `pop`: sono semplici, servono spesso, e con l'esempio accanto si capiscono da soli.

Gli esempi di ogni sezione partono sempre dagli stessi valori, scritti in testa alla sezione. Ogni riga riparte da lì: le modifiche di una riga non passano alla successiva.

Due domande da farsi davanti a ogni metodo, perché è lì che si sbaglia:

- **modifica la struttura, o ne restituisce una nuova?** Le stringhe e le tuple non si modificano mai: i loro metodi restituiscono un valore nuovo, che va assegnato. Molti metodi delle liste e dei dizionari invece modificano la struttura e restituiscono `None`;
- **che cosa succede se l'elemento non c'è?** Alcuni danno un errore, altri restituiscono un valore speciale.

Operatori sulle sequenze

Valgono per stringhe, tuple e liste. Gli esempi usano `s = "pioggia"`.

Operazione	Che cosa fa	Esempio	Risultato
`x[i]`	l'elemento in posizione `i`; il primo ha indice 0	`s[0]`	`'p'`
`x[-i]`	contando dal fondo: `-1` è l'ultimo	`s[-1]`	`'a'`
`x[i:j]`	da `i` a `j`, **`j` escluso**	`s[1:4]`	`'iog'`
`x[:j]` `x[i:]`	dall'inizio fino a `j` escluso · da `i` alla fine	`s[:3]` · `s[3:]`	`'pio'` · `'ggia'`
`x[i:j:passo]`	uno ogni `passo`	`s[::2]`	`'poga'`
`x[::-1]`	al contrario	`s[::-1]`	`'aiggoip'`
`x + y`	concatenazione: una sequenza nuova	`s + '!'`	`'pioggia!'`
`x * n`	ripetizione	`s * 2`	`'pioggiapioggia'`
`e in x`	`True` se `e` c'è; `not in` è il contrario	`'gg' in s`	`True`
`len(x)`	quanti elementi	`len(s)`	`7`

Un indice fuori dalla sequenza dà `IndexError`: l'ultimo indice valido è `len(x) - 1`. Lo slicing invece non dà mai errore: `s[5:100]` restituisce quello che c'è, `'ia'`.

Stringhe

`s = "pioggia"`. **Le stringhe non si modificano**: `s[0] = 'P'` dà `TypeError`, e ogni metodo restituisce una stringa **nuova**. `s.upper()` da solo non cambia `s`: serve `s = s.upper()`.

Metodo	Che cosa fa	Esempio	Risultato
`s.upper()` `s.lower()`	tutto maiuscolo · tutto minuscolo	`s.upper()`	`'PIOGGIA'`
`s.strip()`	toglie spazi e a capo ai due estremi; `lstrip()`, `rstrip()` da un lato solo	`' ciao '.strip()`	`'ciao'`
`s.split()`	spezza sugli spazi, e restituisce una **lista di stringhe**	`'a b c'.split()`	`['a', 'b', 'c']`
`s.split(sep)`	spezza su `sep`	`'3;-1;4'.split(';')`	`['3', '-1', '4']`
`sep.join(lista)`	unisce una lista di stringhe, con `sep` in mezzo	`'-'.join(['a', 'b', 'c'])`	`'a-b-c'`
`s.replace(a, b)`	sostituisce **tutte** le occorrenze di `a` con `b`	`s.replace('g', 'c')`	`'pioccia'`
`s.find(sub)`	posizione della **prima** occorrenza; `-1` se non c'è	`s.find('g')` · `s.find('z')`	`3` · `-1`
`s.index(sub)`	come `find`, ma se non c'è dà errore	`s.index('z')`	`ValueError`
`s.count(sub)`	quante volte compare	`s.count('g')`	`2`
`s.startswith(sub)` `s.endswith(sub)`	`True` se comincia · finisce così	`s.startswith('pio')`	`True`

I pezzi di `split` sono **stringhe**, anche quando contengono cifre: `'-1'` non è un numero. Si convertono con `int()` o `float()`, come quello che arriva da `input()`.

Tuple

`t = (45.47, 9.19)`. **Le tuple non si modificano**: niente `append`, niente `t[0] = ...`. Valgono gli operatori sulle sequenze, più due metodi.

Operazione	Che cosa fa	Esempio	Risultato
`(a, b, ...)`	costruisce una tupla: è la **virgola** a farla, le parentesi aiutano a leggere	`(5,)`	`(5,)`
`(a)`	**non** è una tupla: è `a` fra parentesi	`type((5))`	`<class 'int'>`

Operazione	Che cosa fa	Esempio	Risultato
`x, y = t`	**spacchetta**: un nome per elemento	`lat, lon = t`	`lat` vale `45.47`
`t.index(x)`	posizione della prima occorrenza; `ValueError` se non c'è	`t.index(9.19)`	`1`
`t.count(x)`	quante volte compare	`t.count(45.47)`	`1`
`t + (x,)`	una tupla **nuova** con un elemento in più	`t + (100,)`	`(45.47, 9.19, 100)`

Lo spacchettamento scambia due variabili in una riga: `a, b = b, a`. Il numero di nomi a sinistra deve essere uguale al numero di elementi: altrimenti `ValueError`.

Una tupla può fare da **chiave di un dizionario**, una lista no: `pedine[(0, 3)] = 'rosso'` funziona, `pedine[[0, 3]]` dà `TypeError`.

Liste

`l = [3, 1, 2]`. **Le liste si modificano**: i metodi della prima tabella cambiano `l` e restituiscono `None`. Per questo **`l = l.sort()` è un errore**: ordina la lista, e poi assegna a `l` il valore `None`.

Metodi che modificano la lista

Metodo	Che cosa fa	Esempio	`l` diventa
`l.append(x)`	aggiunge **un** elemento in fondo, qualunque cosa sia	`l.append(4)`	`[3, 1, 2, 4]`
	… anche una lista, che entra come un elemento solo	`l.append([4, 5])`	`[3, 1, 2, [4, 5]]`
`l.extend(seq)`	aggiunge in fondo, **uno per uno**, gli elementi di `seq`	`l.extend([4, 5])`	`[3, 1, 2, 4, 5]`
	… e una stringa entra lettera per lettera	`l.extend('ab')`	`[3, 1, 2, 'a', 'b']`
`l.insert(i, x)`	inserisce `x` in posizione `i`; gli altri scivolano avanti	`l.insert(0, 9)`	`[9, 3, 1, 2]`
`l.remove(x)`	toglie la **prima** occorrenza del **valore** `x`; gli altri scivolano indietro. `ValueError` se `x` non c'è	`l.remove(1)`	`[3, 2]`
`l.pop(i)`	toglie l'elemento in **posizione** `i`, e lo **restituisce**	`l.pop(0)`	`[1, 2]`, restituisce `3`
`l.pop()`	senza argomento, toglie e restituisce l'**ultimo**	`l.pop()`	`[3, 1]`, restituisce `2`

Metodo	Che cosa fa	Esempio	`l` diventa
`del l[i]`	toglie l'elemento in posizione `i`. È un'istruzione, non un metodo	`del l[0]`	`[1, 2]`
`del l[i:j]`	toglie una fetta	`del l[0:2]`	`[2]`
`l[i] = x`	sostituisce l'elemento in posizione `i`	`l[0] = 7`	`[7, 1, 2]`
`l.sort()`	ordina, dal più piccolo	`l.sort()`	`[1, 2, 3]`
`l.sort(reverse=True)`	ordina, dal più grande	`l.sort(reverse=True)`	`[3, 2, 1]`
`l.reverse()`	rovescia l'ordine	`l.reverse()`	`[2, 1, 3]`

Per togliere: **`remove` cerca un valore, `pop` e `del` una posizione**. Solo `pop` restituisce quello che toglie.

Metodi che non modificano la lista

Metodo	Che cosa fa	Esempio	Risultato
`l.index(x)`	posizione della prima occorrenza; `ValueError` se non c'è	`l.index(2)`	`2`
`l.count(x)`	quante volte compare	`l.count(3)`	`1`
`l.copy()`	una lista **nuova** con gli stessi elementi; come `list(l)` e `l[:]`	`l.copy()`	`[3, 1, 2]`

Dizionari

`d = {'lun': 3, 'mar': -1}`. Un dizionario associa **chiavi** a **valori**: non ha posizioni, `d[0]` cerca la chiave `0`. Le chiavi sono **uniche** e **immutabili** (stringhe, numeri, tuple); i valori possono essere qualunque cosa, anche liste o altri dizionari.

Operazione	Che cosa fa	Esempio	Risultato
`d[k]`	il valore della chiave `k`; **`KeyError`** se non c'è	`d['lun']` · `d['dom']`	`3` · `KeyError`
`d[k] = v`	se `k` non c'è la **aggiunge**, se c'è ne **sostituisce** il valore	`d['mer'] = 2` · `d['lun'] = 4`	`d` diventa `{'lun': 3, 'mar': -1, 'mer': 2}` · `{'lun': 4, 'mar': -1}`

Operazione	Che cosa fa	Esempio	Risultato
`d.get(k)`	il valore di `k`; `None` se non c'è, **senza errore**	`d.get('dom')`	`None`
`d.get(k, x)`	il valore di `k`; `x` se non c'è	`d.get('dom', 0)`	`0`
`del d[k]`	toglie la coppia; `KeyError` se non c'è	`del d['lun']`	`d` diventa `{'mar': -1}`
`d.pop(k)`	toglie la coppia, e restituisce il valore	`d.pop('lun')`	`3`; `d` diventa `{'mar': -1}`
`k in d`	`True` se `k` è una **chiave**: non guarda i valori	`'lun' in d` · `3 in d`	`True` · `False`
`d.keys()`	le chiavi	`d.keys()`	`dict_keys(['lun', 'mar'])`
`d.values()`	i valori	`d.values()`	`dict_values([3, -1])`
`d.items()`	le coppie `(chiave, valore)`	`d.items()`	`dict_items([('lun', 3), ('mar', -1)])`
`len(d)`	quante coppie	`len(d)`	`2`

`keys()`, `values()` e `items()` restituiscono una vista, non una lista: si scorrono con un `for`, ma non si indicizzano (`d.keys()[0]` dà `TypeError`), e **seguono il dizionario** se cambia dopo. Per avere una lista, ferma a quel momento: `list(d.keys())`.

L'ordine è quello di inserimento: le chiavi escono nell'ordine in cui sono entrate. Sostituire il valore di una chiave non la sposta.

Contare con un dizionario, in una riga:

```
conteggio[x] = conteggio.get(x, 0) + 1
```

la prima volta `get` restituisce `0` e la chiave nasce con `1`; le volte dopo si incrementa.

Funzioni predefinite

Valgono su tutte le strutture, o quasi. Gli esempi usano `s = "pioggia"`, `l = [3, 1, 2]`, `t = (45.47, 9.19)`, `d = {'lun': 3, 'mar': -1}`.

Funzione	Che cosa fa	Esempio	Risultato
`len(x)`	quanti elementi; per un dizionario, quante coppie	`len(d)`	`2`

Funzione	Che cosa fa	Esempio	Risultato
`sorted(x)`	una lista **nuova**, ordinata; `x` non cambia	`sorted(l)` · `sorted(s)`	`[1, 2, 3]` · `['a', 'g', 'g', 'i', 'i', 'o', 'p']`
`sorted(x, reverse=True)`	ordinata dal più grande	`sorted(l, reverse=True)`	`[3, 2, 1]`
`min(x)` `max(x)`	il più piccolo · il più grande; `ValueError` se `x` è vuota	`min(l)` · `max(s)`	`1` · `'p'`
`sum(x)`	la somma, se gli elementi sono numeri	`sum(l)` · `sum(t)`	`6` · `54.66`
`list(x)`	una lista con gli elementi di `x`	`list(s)` · `list(d)`	`['p', 'i', 'o', 'g', 'g', 'i', 'a']` · `['lun', 'mar']`
`tuple(x)`	una tupla con gli elementi di `x`	`tuple(l)`	`(3, 1, 2)`
`dict(coppie)`	un dizionario da una sequenza di coppie	`dict([('a', 1), ('b', 2)])`	`{'a': 1, 'b': 2}`
`range(stop)`	gli interi da 0 a `stop`, **`stop` escluso**	`list(range(4))`	`[0, 1, 2, 3]`
`range(start, stop, passo)`	da `start`, uno ogni `passo`; con il passo negativo si scende	`list(range(2, 8, 2))` · `list(range(5, 0, -1))`	`[2, 4, 6]` · `[5, 4, 3, 2, 1]`
`enumerate(x)`	le coppie `(indice, elemento)`	`list(enumerate(['x', 'y']))`	`[(0, 'x'), (1, 'y')]`
`enumerate(x, inizio)`	lo stesso, contando da `inizio`	`list(enumerate(['x', 'y'], 1))`	`[(1, 'x'), (2, 'y')]`
`zip(x, y)`	le coppie degli elementi nella stessa posizione; **si ferma alla più corta**, senza avvisare	`list(zip('ab', [1, 2, 3]))`	`[('a', 1), ('b', 2)]`
`dict(zip(chiavi, valori))`	un dizionario da due liste parallele	`dict(zip(['a', 'b'], [1, 2]))`	`{'a': 1, 'b': 2}`

`range`, `enumerate` e `zip` non costruiscono una lista: producono gli elementi **uno alla volta**, mentre il `for` li chiede. Per vederli tutti insieme si passano a `list()`, come negli esempi.

Che cosa visita il `for`

`for elemento in ...`	`elemento` vale, a ogni giro
una stringa	un carattere
una lista o una tupla	un elemento

`for elemento in ...`	`elemento` vale, a ogni giro
un dizionario	una **chiave**
`d.values()`	un valore
`d.items()`	una coppia `(chiave, valore)`: si spacchetta con `for k, v in d.items():`
`range(...)`	un intero
`enumerate(x)`	una coppia `(indice, elemento)`: `for i, e in enumerate(x):`
`zip(x, y)`	una coppia: `for a, b in zip(x, y):`

Le trappole

Un secondo nome non è una copia. `b = a` dà alla stessa lista un secondo nome: modificarla attraverso `b` la modifica anche per `a`. Una copia si fa con `a.copy()` o `list(a)`.

La copia di una lista di liste copia un livello solo. `list(tabella)` è una lista nuova, ma le sue righe sono le stesse di prima: cambiare `copia[0][0]` cambia anche `tabella[0][0]`. Per copiare anche le righe:

```
copia = []
for riga in tabella:
    copia.append(list(riga))
```

`[[0] * 3] * 3` è la stessa riga tre volte. Una griglia si costruisce con una riga nuova a ogni giro:

```
griglia = []
for riga in range(3):
    griglia.append([0] * 3)
```

Non si toglie da una struttura mentre la si scorre. Su una lista, `remove` dentro un `for` fa saltare un elemento a ogni rimozione, senza errori; su un dizionario dà `RuntimeError`. Si costruisce invece una struttura nuova con quello che si vuole tenere.

Due liste parallele si scompagnano. Se `nomi[i]` e `valori[i]` vanno insieme, ordinare una delle due rompe le coppie. Quando due dati stanno insieme, mettili insieme: in un dizionario, o in una lista di tuple.

`input()` e `split()` danno stringhe. Ordinate, finiscono in ordine alfabetico: `['100', '12', '25', '9']`. Si convertono subito, con `int()` o `float()`.